EKWE TECHNOLOGIES LIMITED

Ekwe Protocol

A Decentralized Event-Driven Edge Network
for Resilient Push Messaging and Offline Transactions
Anchored to a Bitcoin Layer-2

David Nzagha

Ekwe Technologies Limited · Nzagha Ventures Limited

Version 1.0 · February 2026

"The ancestral drum of the future.
A sovereign event-driven messaging network powered by people, proof, and Bitcoin."

Abstract

Mission-critical services in regions with intermittent connectivity require reliable dissemination primitives that do not assume continuous Internet access or centralized operators. Contemporary push-notification infrastructures (Apple Push Notification service [APNs] and Firebase Cloud Messaging [FCM]) provide scalable targeted delivery but fundamentally depend on always-reachable cloud brokers and stable IP connectivity [2]. Unstructured Supplementary Service Data (USSD) remains a dominant access channel for mobile financial services in African markets because it operates over cellular signaling channels without requiring an Internet data plan; however, USSD is session-bound to mobile network operator infrastructure, offers limited programmability, and fails under cellular outages [4].

This paper specifies **Ekwe Protocol**, a decentralized, event-driven edge computing network designed for mission-critical operations in challenged environments. The protocol provides a "push-like" messaging substrate supporting offline-first delivery, multi-radio relaying (Bluetooth Low Energy, Wi-Fi peer-to-peer, LoRaWAN, and optional legacy radio), and programmable event execution with verifiable computation. Ekwe adopts an asynchronous message-and-fee model inspired by The Open Network (TON) [10], enabling per-event gas accounting and incentive-compatible reward splitting across relayers, executors, and chain committers. For global settlement and auditability, Ekwe targets a Bitcoin Layer-2 (L2) design using periodic commitments to Bitcoin, leveraging recent work on Bitcoin L2 design patterns [12] and BitVM-style bridging [13].

The paper contributes: (I) a system model and protocol architecture for decentralized push delivery under disruption; (II) a transport-agnostic event format and routing/relay logic tailored to store-and-forward edge networks; (III) a TON-inspired gas and reward mechanism aligned with Decentralized Physical Infrastructure Networks (DePIN); (IV) a dual-token economic model ($EKWE utility token and BTC mining rewards); and (V) an implementation and evaluation blueprint for the flagship offline-finance use case, EkwePay.

Keywords: Decentralized Physical Infrastructure Networks (DePIN), Delay/Disruption-Tolerant Networking (DTN), Push Notifications, USSD, LoRaWAN, Bluetooth Mesh, Wi-Fi Direct, Zero-Knowledge Proofs (ZKP), Verifiable Computing, Bitcoin Layer-2, Incentive Mechanisms, Edge Computing.

Contents

1 Introduction and Vision **4**
1.1 The Ekwe Drum: Cultural Foundation . . . 4
1.2 Contributions . . . 4

2 Background and Motivation **5**
2.1 Centralized Push Notification Infrastructures . . . 5
2.2 USSD and Low-Internet Financial Access . . . 5
2.3 Challenged Networks and DTN . . . 5
2.4 DePIN and Community-Operated Coverage . . . 5
2.5 Multi-Radio Edge Transports . . . 6

3 Problem Statement **6**

4 Design Goals and Non-Goals **6**
4.1 Goals . . . 6
4.2 Non-Goals (v0) . . . 7

5 System Architecture **7**

6 Node Roles and Identity **8**
6.1 Identity Model . . . 8
6.2 Node Types . . . 8
6.3 Network Assumptions . . . 8

7 Ekwe Event Layer **10**
7.1 Event Format . . . 10
7.2 Addressing and Subscription . . . 10
7.3 Dissemination Semantics . . . 11
7.3.1 Delivery Probability Model . . . 11

8 Bridge Logic (Ekwe Edge Agent) **13**
8.1 Functional Responsibilities . . . 13
8.2 Convergence Layer Abstraction . . . 13
8.3 Smartphone Relaying . . . 13

9 Transport Layer **13**

10 Gas and Incentive Model **14**
10.1 Rationale . . . 14
10.2 Fee Decomposition . . . 14
10.2.1 Fee Computation . . . 14
10.3 Sponsorship Model . . . 15
10.4 Reward Splitting . . . 15

10.4.1 Per-Relayer Distribution . . . 16

11 Token Economics **16**
11.1 The $EKWE Utility Token . . . 16
11.1.1 Staking Requirements . . . 17
11.1.2 Slashing . . . 17
11.2 BTC Mining Rewards . . . 17

12 Execution and Verifiable Computing **18**
12.1 Executable Events . . . 18
12.2 Proof of Execution via zkVM . . . 18
12.3 Execution Runtimes . . . 19

13 Bitcoin L2 Anchoring and Settlement **19**
13.1 Design Positioning . . . 19
13.2 Bridging and Dispute Framing . . . 19
13.3 Settlement for EkwePay . . . 19
13.4 Chain Design Options . . . 19

14 EkwePay: Flagship Use Case **20**
14.1 Transaction Workflow . . . 20
14.2 Offline Risk Controls . . . 21
14.2.1 Risk Bound Formalization . . . 21

15 Hardware: EkweBox and Smartbox **21**
15.1 EkweBox . . . 21
15.2 Smartbox . . . 22

16 SDK and Developer Toolkit **22**
16.1 SDK Architecture . . . 22
16.2 Core API Surface . . . 22
16.3 Transport Abstraction . . . 23
16.4 Developer Workflow . . . 23
16.5 Gas Sponsorship for dApps . . . 24

17 Security and Privacy **24**
17.1 Identity and Authentication . . . 24
17.2 Replay, Spam, and Flooding Resistance . . . 24
17.3 Sybil Resistance . . . 24
17.4 Privacy and Metadata Minimization . . . 25

18 Implementation Blueprint and Evaluation Methodology **25**
18.1 Reference Implementation . . . 25
18.2 Evaluation Metrics . . . 25
18.3 Evaluation Scenarios . . . 26

18.4 Baselines 26

19 Roadmap **27**

20 Limitations and Open Problems **27**

21 Conclusion **28**

1. Introduction and Vision

Digital public infrastructure increasingly assumes uninterrupted connectivity. In practice, connectivity is uneven, fragile, and frequently disrupted by power constraints, backhaul failures, congestion, or deliberate shutdowns. Under these conditions, critical operations (payments, medical logistics, emergency alerts, and safety coordination) degrade sharply when their communication substrate depends on centralized Internet services.

Push notification systems are a canonical example. APNs and FCM provide a standardized interface for applications to deliver targeted notifications to devices at global scale, but delivery requires a provider server to reach a centralized push service over the Internet [2]. When connectivity is intermittent or absent, push delivery collapses into delayed polling, manual coordination, or complete silence.

USSD occupies a different niche. Defined for public land mobile networks (PLMNs) as a session-oriented mechanism enabling interactive exchanges between a mobile station and operator-defined applications via signaling channels [4], USSD is widely used for mobile money and merchant transactions [6]. However, USSD is structurally constrained by the operator's signaling plane, limited message complexity, and the requirement of at least partial cellular service. It is therefore "offline from the Internet," but not offline from the network.

1.1 The Ekwe Drum: Cultural Foundation

Ekwe Protocol is motivated by an older idea with modern constraints: the *Ekwe*, a traditional communication drum used by Igbo communities in West Africa, as an always-on, community-scaled dissemination mechanism. The Ekwe drum served as a decentralized broadcast network: messages were encoded in tonal patterns, relayed across villages by drummers, and received by anyone within earshot. No central authority controlled the channel; any community member with an Ekwe could originate, relay, or respond to messages.

This ancestral model maps directly to the protocol's architecture:

- **Drummers** → **Relay Nodes**: community members who forward messages.
- **Drum patterns** → **Ekwe Events**: structured, signed message envelopes.
- **Village squares** → **Edge clusters**: local broadcast domains.
- **Inter-village relay chains** → **Multi-hop store-and-forward**: DTN-style propagation.

Ekwe Protocol aims to provide a decentralized, resilient, event-driven edge network that supports: (a) push-like delivery without continuous Internet; (b) local relaying by community-operated devices (including smartphones); (c) programmable event execution at the edge; and (d) cryptographic accountability and incentive alignment via a Bitcoin L2.

1.2 Contributions

The contributions of this paper are fivefold:

1. A system architecture for decentralized push-style dissemination over heterogeneous transports with store-and-forward semantics.
2. A protocol-level event model supporting targeted delivery, scoped broadcast, subscription filtering, and replay resistance.

3. A TON-inspired gas and reward splitting mechanism spanning forwarding, computation, and storage resources [11].
4. A dual-token economic model combining the $EKWE utility token with Bitcoin mining rewards through dedicated hardware (EkweBox/Smartbox).
5. An execution and settlement pipeline for verifiable event execution and delayed anchoring to Bitcoin L2 [12].

2. Background and Motivation

2.1 Centralized Push Notification Infrastructures

APNs uses provider-to-APNs connections (HTTP/2 with token-based authentication) to deliver notifications to Apple devices, while FCM provides an HTTP v1 API with topic/token addressing to route messages to Android and cross-platform clients [2, 3]. Both achieve scale by centralizing delivery, which implies: (I) continuous Internet reachability from the provider; and (II) dependence on a trusted third-party broker for targeting, routing, and queuing.

2.2 USSD and Low-Internet Financial Access

USSD is standardized for PLMNs as a mechanism enabling dialogue-style interactions carried over signaling channels rather than IP data bearers [4, 5]. In practice, USSD is a major bearer channel for mobile money, with a large fraction of merchant transactions relying on USSD due to its compatibility with feature phones and minimal data requirements [6]. USSD's constraints are well understood: sessions are operator-mediated, state is maintained within the session boundary, and the model offers limited programmability.

Ekwe Protocol is motivated by the observation that neither centralized push nor USSD provides a neutral, community-operated substrate that remains functional when both Internet backhaul and operator signaling are impaired.

2.3 Challenged Networks and DTN

Delay/Disruption-Tolerant Networking (DTN) explicitly addresses environments lacking stable end-to-end paths by adopting store-and-forward routing and overlay naming [1]. Ekwe adopts DTN's core insight (*progress is possible without continuous connectivity*) and applies it to modern mobile/IoT radios and application semantics, adding targeted "push-like" semantics, economic metering, and optional verifiable execution.

2.4 DePIN and Community-Operated Coverage

Decentralized Physical Infrastructure Networks (DePIN) denote networks where cryptographic incentives coordinate deployment and operation of physical infrastructure [16]. Helium provides a prominent precedent for community-operated LoRaWAN-style coverage with token incentives, illustrating both feasibility and design tradeoffs [15]. Ekwe targets a broader primitive than "coverage": reliable event dissemination and execution with explicit gas accounting.

2.5 Multi-Radio Edge Transports

Ekwe targets a pragmatic transport palette already available in consumer and low-cost hardware:

- **LoRaWAN**: long-range, low-throughput communication optimized for battery-powered devices [7].
- **Bluetooth Mesh**: many-to-many relaying and group addressing for local dissemination over BLE [8].
- **Wi-Fi Direct (P2P)**: higher-throughput peer-to-peer connectivity without an access point, widely available on smartphones [9].
- **Legacy radio** (UHF/VHF), NFC, QR codes, and audio tones for extreme offline scenarios.

3. Problem Statement

The problem addressed is the absence of an open, self-hostable, incentive-compatible alternative to centralized push infrastructures that can:

(a) Deliver targeted and broadcast "push-like" events under intermittent connectivity;
(b) Operate over heterogeneous local radios without requiring cellular networks;
(c) Support application-defined execution (e.g., payment authorization workflows) at the edge;
(d) Provide cryptographic accountability and economically aligned participation for relayers and validators; and
(e) Enable eventual settlement and auditability through Bitcoin L2 anchoring.

4. Design Goals and Non-Goals

4.1 Goals

G1 **Offline-first delivery**: events should propagate and be deliverable without Internet backhaul, and in some deployments without cellular coverage.

G2 **Transport agnosticism**: a common event layer must run across BLE, Wi-Fi P2P, LoRaWAN, and optional legacy radios [8].

G3 **Incentive compatibility**: participants should be compensated for measurable fulfillment contributions.

G4 **Verifiable execution**: where events trigger state transitions, execution should be provable [14].

G5 **Bitcoin-anchored auditability**: periodic commitments to Bitcoin should provide high-integrity settlement [12].

G6 **Edge-first computing**: all computation happens at the edge; cloud is optional.

G7 **People-powered infrastructure**: any mobile device can join the network as a node.

G8 **Privacy by design**: zero-knowledge proofs enable verifiable actions without revealing data.

4.2 Non-Goals (v0)

N1 Guaranteed real-time delivery under all adversarial conditions.

N2 Replacing cellular Internet; Ekwe complements it and degrades gracefully.

N3 Solving all offline double-spend risk without any synchronization; instead, EkwePay is designed with bounded risk and eventual settlement constraints.

5. System Architecture

Ekwe Protocol is defined as a layered system with explicit separation between transport adapters and the event layer. Figure 1 illustrates the five-layer stack.

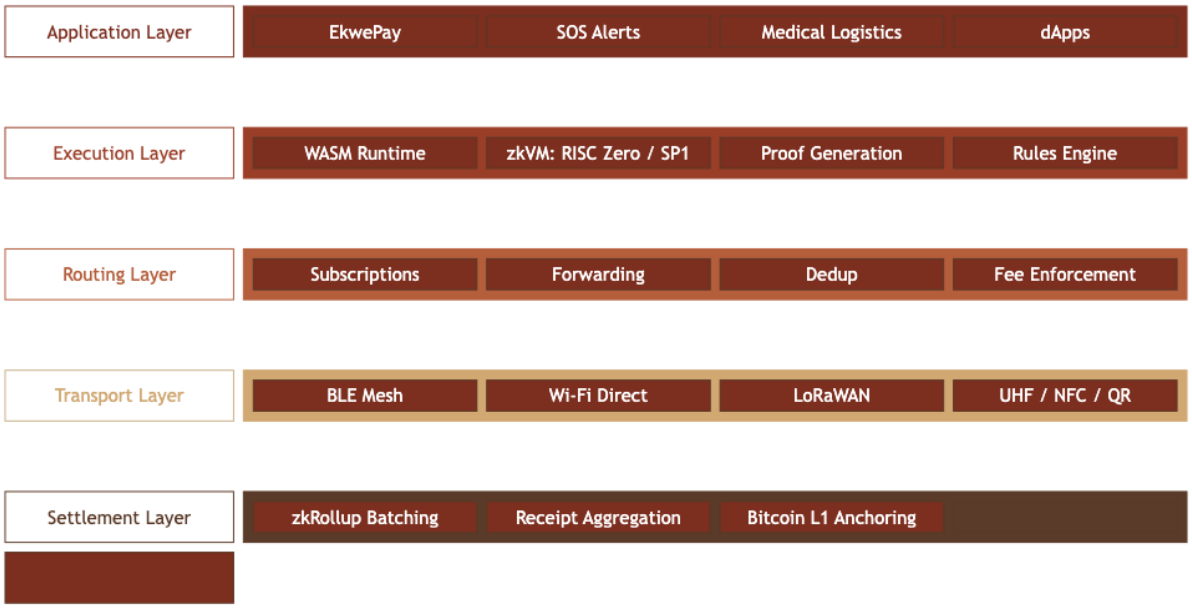


Figure 1: Ekwe Protocol layered architecture.

1. **Application Layer**: mission-critical services including EkwePay (offline payments), SOS emergency alerts, medical logistics, and third-party dApps built on the Ekwe SDK.
2. **Execution Layer**: optional deterministic execution of event payloads (WASM modules) and receipt/proof generation via zkVM [14].
3. **Event Routing Layer (Bridge Logic)**: subscription management, forwarding policy, deduplication, fee enforcement, receipt handling, and store-and-forward caching.
4. **Transport Layer**: heterogeneous radio adapters (BLE mesh, Wi-Fi P2P, LoRaWAN, and optional legacy interfaces) [7, 8, 9].
5. **Settlement Layer**: Bitcoin L2 commitments with periodic anchoring and dispute pathways [12, 13].

6. Node Roles and Identity

6.1 Identity Model

Each node in the Ekwe network possesses:

- A **Node ID** in Decentralized Identifier (DID) format.
- An **Ed25519 public/private key pair** for signing and verification.
- A **role designation**: Mobile Node, Relay Node, Execution Node, Commit Node, or Smartbox Node.
- **Message filters**: topics, regions, and tags defining subscription interests.
- A **token balance** for spending gas or receiving rewards.

6.2 Node Types

Table 1: Ekwe Protocol node types and capabilities.

Node Type	Description
Mobile Node	Smartphone running an Ekwe Edge Agent; capable of subscribing, sending, and optionally relaying events. First-class participant due to ubiquity and multi-radio support.
Light Relay	Phone or embedded device providing short-term message caching and BLE/Wi-Fi forwarding with minimal resource commitment.
Full Relay	LoRa+Wi-Fi+ZK capable node providing persistent store-and-forward, consensus participation, and higher QoS guarantees.
Execution Node	Authorized to execute event payloads (WASM or zkVM modules) and produce verifiable receipts/proofs.
Commit Node	Aggregates receipts and posts state commitments to the Bitcoin L2 and periodically to Bitcoin L1.
EkweBox	Purpose-built hardware node: solar/AC powered, multi-radio (LoRa + BLE + Wi-Fi), persistent storage. Designed for permanent community relay infrastructure.
Smartbox	Advanced EkweBox variant incorporating Interaction Combinator-based ASIC mining hardware, enabling Bitcoin mining alongside event routing.

6.3 Network Assumptions

1. Partitions are common; end-to-end paths cannot be assumed.
2. Nodes are heterogeneous in power, bandwidth, and availability.
3. Adversarial behavior is possible (spam floods, selective forwarding, replay).

4. A subset of nodes periodically gains Internet access, enabling delayed global synchronization.

Figure 2 illustrates how these node types interconnect across edge clusters via heterogeneous radio links.

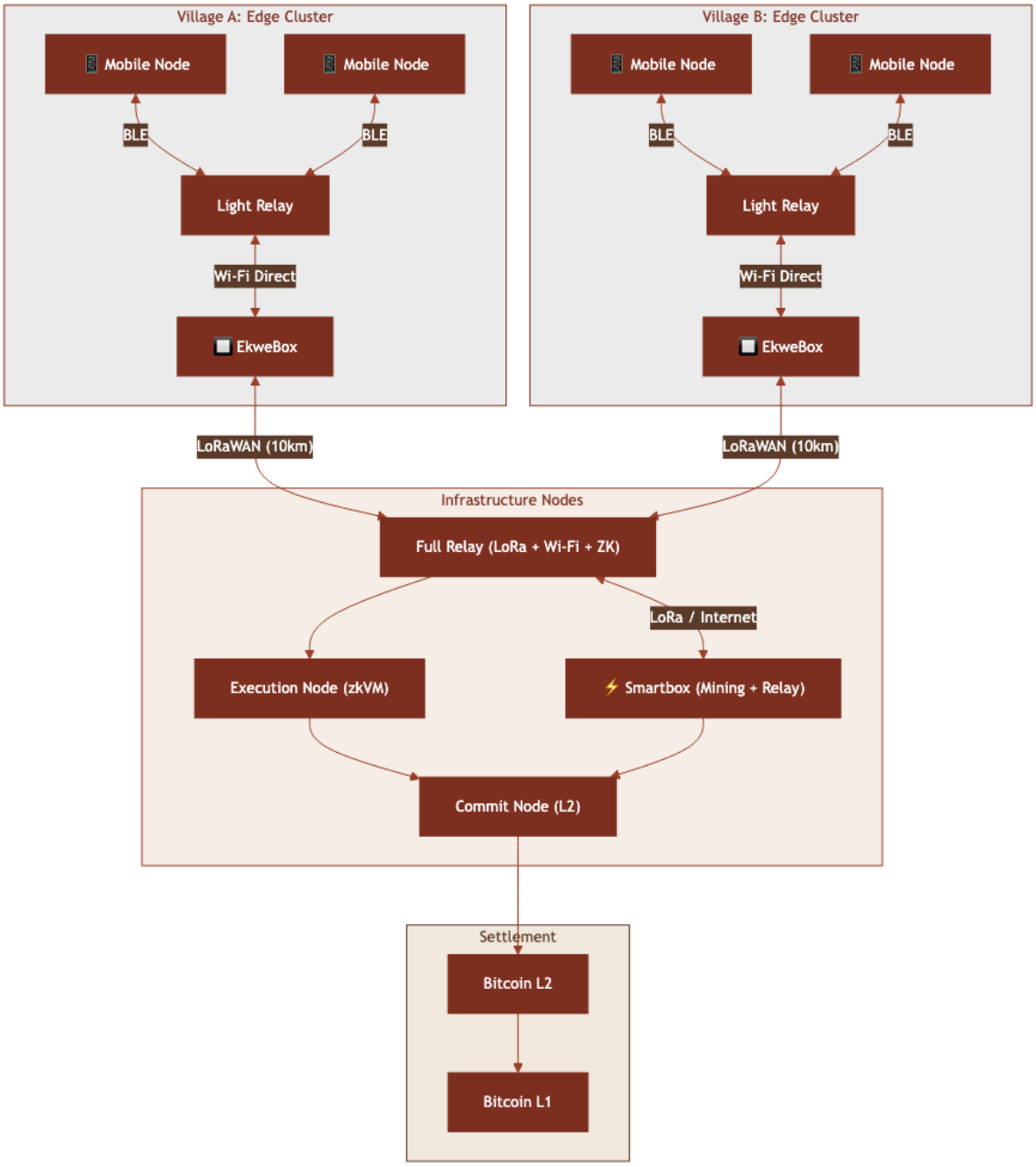


Figure 2: Ekwe network topology: edge clusters connected via multi-radio relay infrastructure.

7. Ekwe Event Layer

7.1 Event Format

Ekwe Protocol is specified around a compact, signed event envelope. The canonical representation is shown below; serialization may use CBOR or Protocol Buffers for constrained links, with custom compact encodings for LoRa.

Listing 1: Ekwe Event envelope (conceptual schema).

```
EkweEvent := {
  event_id        : UUID,
  version         : uint8,
  timestamp       : ISO8601,
  nonce           : uint64,
  from_node_id    : DID,
  to_address      : Address (unicast | multicast | broadcast),
  topic           : string,
  scope           : ScopeDescriptor,
  ttl             : uint16,
  qos_class       : enum { BEST_EFFORT, RELIABLE, CRITICAL },
  payload_type    : enum { DATA, EXECUTABLE, REFERENCE },
  payload         : bytes (inline or content-addressed ref),
  fee             : { f_fwd, f_cmp, f_str },
  fulfillment_policy : FulfillmentPolicy,
  relay_trace     : [NodeID],
  signature       : Ed25519Signature,
  optional_proof  : ZKProof | null,
  optional_receipt_ref : Hash | null
}
```

Key design decisions:

- `to_address` supports unicast (node), multicast (group), and scoped broadcast.
- `topic` enables pub/sub-style filtering for push-like delivery.
- `ttl` bounds propagation cost and limits stale relays, analogous to TTL concepts in mesh systems [8].
- `nonce` and `event_id` provide replay resistance and deduplication.
- `fee` fields enable per-event gas accounting (Section 10).

7.2 Addressing and Subscription

Ekwe supports three addressing modes:

1. **Unicast**: direct node addressing using DID-like identifiers.
2. **Multicast**: group identifiers (e.g., `clinic:ward-3`, `market:zone-2`).
3. **Scoped broadcast**: geofenced region codes or transport-local broadcast.

Nodes register subscriptions locally in the Edge Agent. When possible, subscription commitments are published to the L2 for accountability in paid delivery contexts.

7.3 Dissemination Semantics

Ekwe adopts DTN-aligned store-and-forward semantics [1]. Each relay maintains:

- A bounded cache of recent `event_id` values for deduplication.
- A local "interest table" derived from neighbor subscriptions and observed demand.
- A forwarding policy dependent on `qos_class`, TTL, fee sufficiency, and local congestion.

QoS Classes:

- **Class 0 (Best-effort)**: opportunistic forwarding, minimal redundancy.
- **Class 1 (Reliable)**: replication factor $r > 1$ across diverse next-hops when available.
- **Class 2 (Critical)**: priority scheduling, wider replication, and optional acknowledgment aggregation. Reserved for emergency broadcasts and high-value transactions.

7.3.1 Delivery Probability Model

For a QoS class q event with replication factor $r(q)$ and TTL T, we model delivery probability over a network of N nodes with average contact rate λ_c (pairwise contact events per unit time). Under the epidemic routing model with bounded replication:

$$P_{\text{deliver}}(T, r, N) = 1 - \left(1 - \frac{r}{N}\right)^{\lambda_c \cdot T \cdot N} \tag{1}$$

For sparse networks ($r \ll N$), this simplifies to:

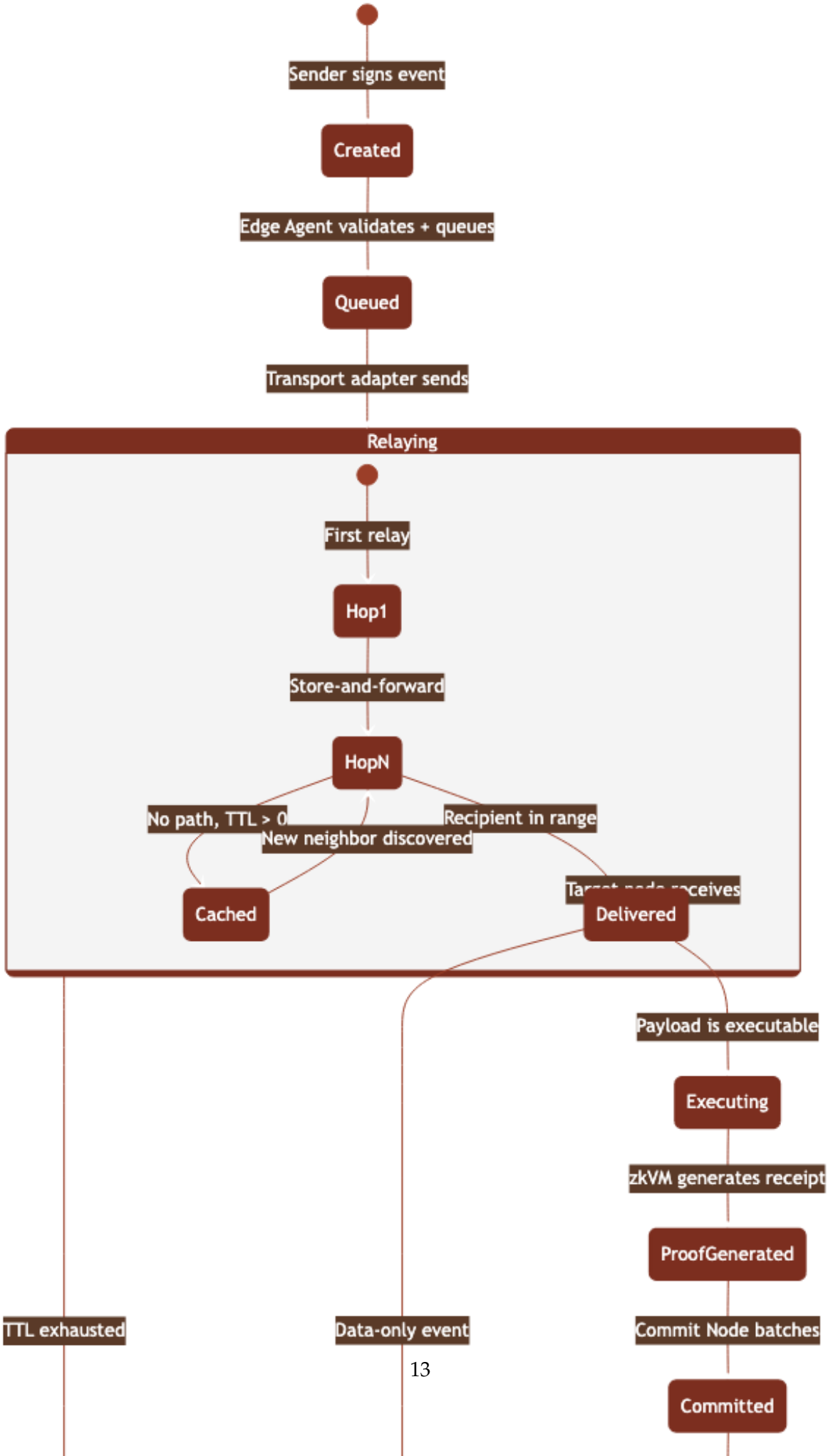
$$P_{\text{deliver}} \approx 1 - e^{-\lambda_c \cdot r \cdot T} \tag{2}$$

The replication factor $r(q)$ is QoS-dependent:

$$r(q) = \begin{cases} 1 & \text{if } q = 0 \\ \lceil \log_2(N_{\text{local}}) \rceil & \text{if } q = 1 \\ N_{\text{local}} & \text{if } q = 2 \end{cases} \tag{3}$$

where N_{local} is the number of reachable next-hop relays. Class 2 events flood the local neighborhood to maximize delivery probability at the cost of higher gas consumption.

Figure 3 shows the complete lifecycle of an Ekwe event from creation through settlement.


Sender signs event
Created
Edge Agent validates + queues
Queued
Transport adapter sends
Relaying
First relay
Hop1
Store-and-forward
HopN
No path, TTL > 0
New neighbor discovered
Recipient in range
Cached
Delivered
Payload is executable
Executing
zkVM generates receipt
ProofGenerated
TTL exhausted
Data-only event
Commit Node batches
Committed

8. Bridge Logic (Ekwe Edge Agent)

The Edge Agent is the protocol's core runtime: a cross-protocol event router and execution engine running on phones, routers, ESP32/Raspberry Pi boards, and custom EkweBox hardware.

8.1 Functional Responsibilities

1. **Transport multiplexing**: selecting BLE, Wi-Fi P2P, LoRaWAN, or other bearers depending on link availability and policy [9].
2. **Event normalization**: mapping transport frames into EkweEvent envelopes.
3. **Policy enforcement**: TTL checks, rate limits, subscription filters, minimum fee validation.
4. **Cryptographic operations**: Ed25519 signing, verification, and receipt handling.
5. **Store-and-forward**: caching with eviction strategies, prioritization, and anti-replay logic.
6. **Execution dispatch**: routing executable payloads to authorized Execution Nodes.

8.2 Convergence Layer Abstraction

Following DTN convergence-layer separation [1], each transport implements a `ConvergenceAdapter` interface:

Listing 2: Convergence Adapter interface.

```
trait ConvergenceAdapter {
  fn discover_neighbors() -> Vec<NeighborDescriptor>;
  fn link_metrics() -> LinkMetrics;  // bandwidth, energy_cost,
     reliability
  fn send(event: &EkweEvent, neighbor: &NeighborDescriptor) -> Result;
  fn receive() -> Option<EkweEvent>;
}
```

This decouples the event layer from transport details, enabling deployment across diverse radios and hardware platforms.

8.3 Smartphone Relaying

Smartphones are first-class relayers due to their ubiquity and multi-radio capabilities. Wi-Fi Direct supports peer-to-peer group formation without an access point, enabling "pop-up" high-throughput sync clusters [9]. BLE and Bluetooth Mesh support local dissemination under low power [8]. LoRaWAN bridges extend range via fixed or semi-fixed relay nodes [7]. Ekwe uses these primitives to approximate push delivery in the absence of Internet by allowing devices to "listen" for relevant topics and addresses locally, receiving events through nearby relays.

9. Transport Layer

Ekwe employs a **transport multiplexer** that provides a unified interface across heterogeneous radio technologies. The multiplexer selects the optimal transport based on availability, link quality, energy budget, and payload characteristics.

Table 2: Transport characteristics and use cases.

Transport	Range	Bandwidth	Primary Use
BLE Mesh	∼50m	Low	Proximity sync, local relay
Wi-Fi Direct	∼200m	High	Bulk sync, cluster formation
LoRaWAN	∼10km	Very Low	Wide-area relay, rural coverage
UHF/VHF Radio	∼50km	Very Low	Emergency, extreme offline
NFC	Contact	Medium	Device pairing, key exchange
QR Codes	Visual	One-shot	Offline payment initiation
Audio Tones	∼10m	Minimal	Extreme fallback

The architectural separation of small control events (transmitted over long-range, low-throughput LoRaWAN) from bulk synchronization phases (executed opportunistically over Wi-Fi Direct) is a central design hypothesis. This enables efficient resource utilization: payment requests and alerts traverse LoRa links, while proof batches and ledger updates synchronize over high-bandwidth local clusters.

10. Gas and Incentive Model

10.1 Rationale

In open relaying networks, anti-spam and incentive alignment are inseparable. Ekwe adopts a per-event gas model that charges for resource consumption and distributes rewards among fulfillment participants. The gas system is *blind* to message type due to zero-knowledge properties; a payment event and an alert event incur fees based on resource usage, not semantic content.

10.2 Fee Decomposition

Inspired by TON's multi-phase fee structure [10, 11], Ekwe decomposes event costs into three components:

1. **Forwarding Fee** (f_{fwd}): proportional to payload size, hop count, and QoS class.
2. **Compute Fee** (f_{cmp}): paid when an Execution Node runs a payload.
3. **Storage Fee** (f_{str}): paid for retaining events/receipts beyond an immediate forwarding window.

10.2.1 Fee Computation

Let e denote an event with payload size $|e|$ (bytes), expected hop count h, QoS class $q \in \{0, 1, 2\}$, compute gas units g_c, and requested storage duration τ_s (seconds). The total fee $F(e)$ is:

$$f_{\text{fwd}}(e) = p_{\text{byte}} \cdot |e| \cdot h \cdot \mu(q) \tag{4}$$

$$f_{\text{cmp}}(e) = p_{\text{gas}} \cdot g_c \tag{5}$$

$$f_{\text{str}}(e) = p_{\text{store}} \cdot |e| \cdot \tau_s \tag{6}$$

$$F(e) = f_{\text{fwd}}(e) + f_{\text{cmp}}(e) + f_{\text{str}}(e) \tag{7}$$

where p_{byte}, p_{gas}, and p_{store} are governance-set base prices denominated in $EKWE, and $\mu(q)$ is the QoS multiplier:

$$\mu(q) = \begin{cases} 1.0 & \text{if } q = 0 \text{ (Best-effort)} \\ 2.0 & \text{if } q = 1 \text{ (Reliable)} \\ 5.0 & \text{if } q = 2 \text{ (Critical)} \end{cases} \tag{8}$$

For events with `payload_type = EXECUTABLE`, compute gas g_c is metered during WASM/zkVM execution. For data-only events, $g_c = 0$.

10.3 Sponsorship Model

Ekwe's primary operational model follows an **integrator-sponsored gas** pathway:

- An **Integrator** (e.g., EkwePay operator, clinic system, logistics platform) prepays or sponsors event gas.
- The integrator may recoup costs from end-users at the application layer via microinvoicing.

This mirrors centralized push economics (operators pay infrastructure costs) while decentralizing the fulfillment path.

10.4 Reward Splitting

Each fulfilled event produces a `FulfillmentReceipt` listing contributing nodes (ordered by `relay_trace` and role). Rewards are split as:

$$R_{\text{total}} = f_{\text{fwd}} + f_{\text{cmp}} + f_{\text{str}} \tag{9}$$

$$R_{\text{relayer}} = \alpha \cdot f_{\text{fwd}} \tag{10}$$

$$R_{\text{executor}} = \beta \cdot f_{\text{cmp}} \tag{11}$$

$$R_{\text{committer}} = \gamma \cdot (f_{\text{fwd}} + f_{\text{cmp}}) \tag{12}$$

$$R_{\text{storage}} = \delta \cdot f_{\text{str}} \tag{13}$$

where $\alpha + \beta + \gamma + \delta = 1$. The parameters $\alpha, \beta, \gamma, \delta$ are governance-controlled, published on-chain, and may be dynamically tuned for network conditions.

10.4.1 Per-Relayer Distribution

When multiple relayers participate in forwarding (as recorded in `relay_trace`), the forwarding reward R_{relayer} is distributed among n relayers. Let w_i denote the weight assigned to relayer i based on link quality and hop position:

$$w_i = \frac{1}{\text{hop}_i} \cdot \lambda_i \tag{14}$$

where hop_i is the relayer's position in the trace (earlier relayers receive more) and $\lambda_i \in (0, 1]$ is a link-quality factor derived from transport reliability metrics. Each relayer receives:

$$R_i = R_{\text{relayer}} \cdot \frac{w_i}{\sum_{j=1}^{n} w_j} \tag{15}$$

Figure 4 illustrates the complete fee decomposition and reward distribution flow.

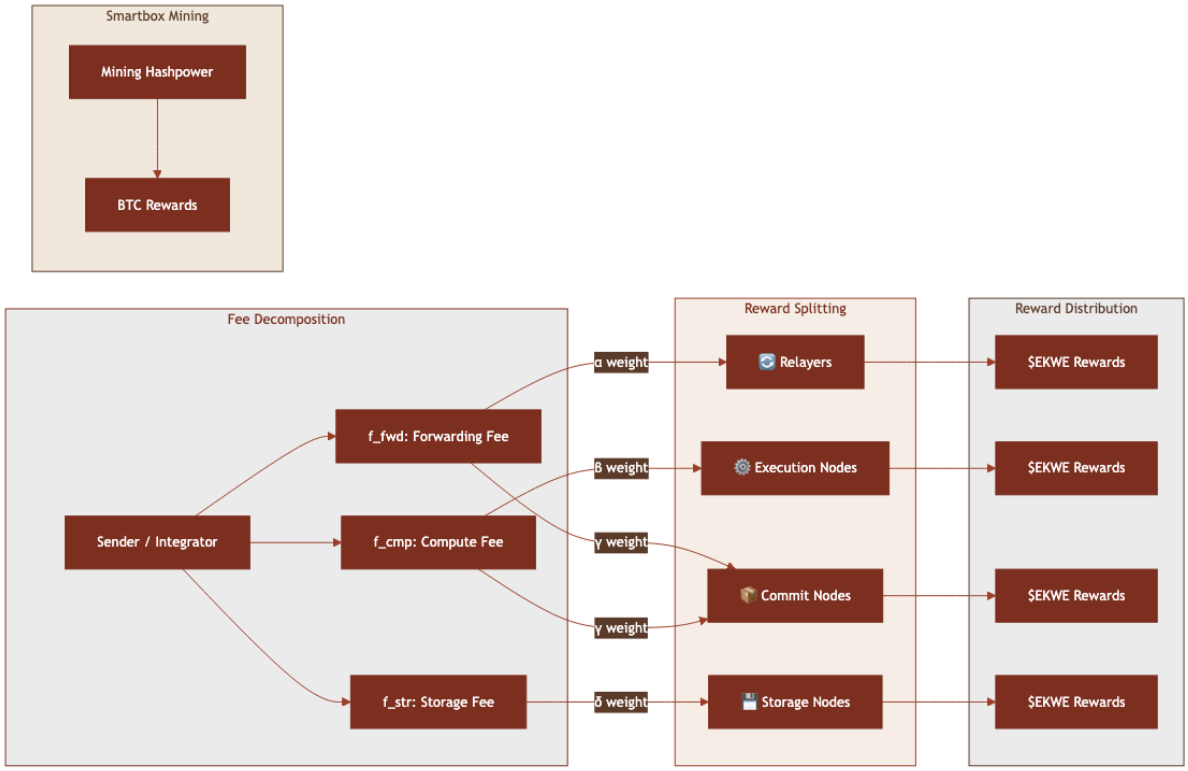


Figure 4: Gas fee decomposition and reward splitting across network participants.

11. Token Economics

Ekwe Protocol employs a **dual-token model** designed to align short-term utility with long-term infrastructure investment.

11.1 The $EKWE Utility Token

$EKWE serves as the native protocol token for:

- **Gas payments**: all event fees are denominated in $EKWE.
- **Node staking**: nodes stake $EKWE to signal commitment and participate in higher-trust roles (Execution, Commit).
- **dApp access**: third-party applications pay $EKWE for protocol integration.
- **Governance**: token-weighted voting on protocol parameters ($\alpha, \beta, \gamma, \delta$, fee floors, QoS policies).

11.1.1 Staking Requirements

Nodes must meet minimum stake thresholds $S_{\min}(r)$ to assume role r:

$$S_{\min}(r) = \begin{cases} 0 & \text{if } r = \text{Mobile Node} \\ S_0 & \text{if } r = \text{Light Relay} \\ k_1 \cdot S_0 & \text{if } r = \text{Full Relay} \\ k_2 \cdot S_0 & \text{if } r = \text{Execution Node} \\ k_3 \cdot S_0 & \text{if } r = \text{Commit Node} \end{cases} \tag{16}$$

where S_0 is the base stake unit and $k_1 < k_2 < k_3$ are role multipliers set by governance. A node's effective reputation score ρ_i is computed as:

$$\rho_i = \frac{s_i}{S_{\min}(r_i)} \cdot \left(\frac{d_{\text{success},i}}{d_{\text{total},i}} \right)^{\eta} \cdot u_i \tag{17}$$

where s_i is the node's actual stake, $d_{\text{success},i}/d_{\text{total},i}$ is its historical delivery ratio, η is a sensitivity exponent, and $u_i \in [0, 1]$ is a normalized uptime factor. Nodes with $\rho_i < \rho_{\min}$ are deprioritized in routing and excluded from high-QoS fulfillment.

11.1.2 Slashing

Misbehavior detected via invalid receipts, selective forwarding proofs, or failed challenges triggers stake reduction:

$$s_i' = s_i - \min\left(\sigma \cdot s_i,\ s_i - S_{\min}(r_i)\right) \tag{18}$$

where $\sigma \in (0, 1)$ is the slashing fraction. Nodes slashed below $S_{\min}$ are automatically demoted.

11.2 BTC Mining Rewards

Smartbox nodes (Section 15) incorporate specialized mining hardware based on Interaction Combinator architectures, enabling energy-efficient Bitcoin mining alongside event routing. BTC rewards are distributed proportionally based on a composite score:

$$\text{BTC}_i = B_{\text{pool}} \cdot \frac{\omega_i}{\sum_{j \in \mathcal{N}} \omega_j} \tag{19}$$

where B_{pool} is the total BTC earned by the mining pool in a reward period, $\mathcal{N}$ is the set of active Smartbox nodes, and ω_i is a composite weight:

$$\omega_i = \phi \cdot H_i + \psi \cdot V_i + \chi \cdot U_i \tag{20}$$

where H_i is hashpower contribution (normalized), V_i is volume of Ekwe events relayed (normalized), U_i is uptime fraction, and $\phi + \psi + \chi = 1$ are governance-set weights. This design ensures that Smartbox operators are incentivized not only for mining but also for maintaining high-quality relay service.

This dual-token structure creates two distinct value streams: $EKWE captures protocol-level utility and governance, while BTC mining rewards provide a hard-money incentive for deploying and maintaining physical infrastructure, bridging the DePIN model with Bitcoin's established monetary properties.

Figure 5 provides an overview of the dual-token model and value flows across network participants.

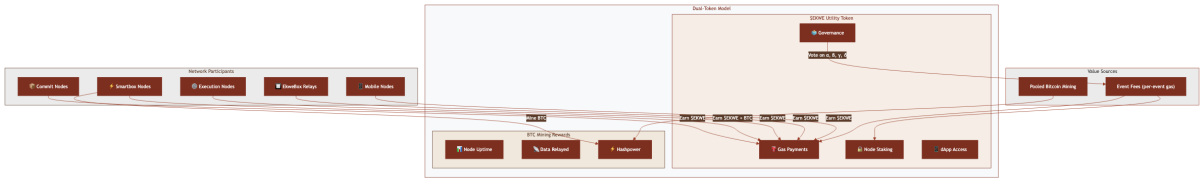


Figure 5: Dual-token economic model: $EKWE utility and BTC mining rewards.

12. Execution and Verifiable Computing

12.1 Executable Events

Some Ekwe events carry logic (or references to logic) requiring deterministic execution:

- Validating a payment authorization policy.
- Performing risk checks and compliance verification.
- Generating state transition witnesses for L2 settlement.
- Executing smart contract logic at the edge.

12.2 Proof of Execution via zkVM

Ekwe targets proof-carrying execution using a zero-knowledge virtual machine (zkVM). RISC Zero provides receipts that prove correct execution of arbitrary code without revealing additional information beyond what is encoded in the receipt [14]. Ekwe's Execution Nodes generate proofs/receipts that are:

1. Locally verifiable by nearby nodes (where feasible).
2. Batch-committable to the L2 for dispute resolution.

In bandwidth-constrained settings, receipts may be generated asynchronously and propagated as follow-on events, allowing dissemination and settlement to proceed even when proof generation is delayed.

12.3 Execution Runtimes

The protocol supports multiple execution backends:

- **WASM**: lightweight, portable execution for general event payloads.
- **zkVM (RISC Zero, SP1)**: for high-assurance proofs on critical operations [14].
- **zkWASM**: combining WASM portability with zero-knowledge proof generation.
- **Rules Engine**: a rule-based router for simple event processing and cache-first logic on resource-constrained nodes.

13. Bitcoin L2 Anchoring and Settlement

13.1 Design Positioning

Bitcoin L2 systems vary widely; a recent Systematization of Knowledge identifies multiple design patterns for constructing L2s via transaction modifications and proof mechanisms [12]. Ekwe adopts a pattern compatible with:

1. Periodic commitments to Bitcoin for auditability.
2. L2 dispute framing that can tolerate delayed connectivity.

13.2 Bridging and Dispute Framing

BitVM2 proposes a paradigm for enabling arbitrary program verification using Bitcoin as a dispute layer, including a bridge design for L2 systems [13]. Ekwe's roadmap is compatible with BitVM-style challenge mechanisms for:

- Bridging BTC into the Ekwe L2.
- Enforcing settlement outcomes for disputed transactions.
- Verifying execution receipts on-chain when challenged.

13.3 Settlement for EkwePay

EkwePay requires a clear separation between:

1. **Local authorization**: user experience under outage, with signed intents and bounded validity windows.
2. **Eventual settlement**: final ledger updates anchored to the L2 and periodically to Bitcoin L1 upon synchronization.

The protocol treats offline approvals as signed intents with bounded validity windows and risk controls, with final ledger updates occurring upon any node regaining backhaul connectivity.

13.4 Chain Design Options

Ekwe's settlement layer is designed to be modular:

- **zkRollup**: batched proofs from Execution Nodes committed to Bitcoin L2.
- **DAG with lazy finality**: lightweight consensus for edge environments.
- **Federation**: initial bootstrap mechanism for early network deployment.

- **Embedded mini-chain**: local SQLite-based chain or zk-rollup buffer for offline operation.

14. EkwePay: Flagship Use Case

EkwePay is the flagship application demonstrating Ekwe Protocol's capabilities for offline-first financial transactions. It differs from USSD-based mobile money in two fundamental ways: (I) USSD requires cellular service continuity [4]; (II) EkwePay operates over community-provisioned local transports with eventual settlement.

14.1 Transaction Workflow

Figure 6 illustrates the complete EkwePay transaction flow.

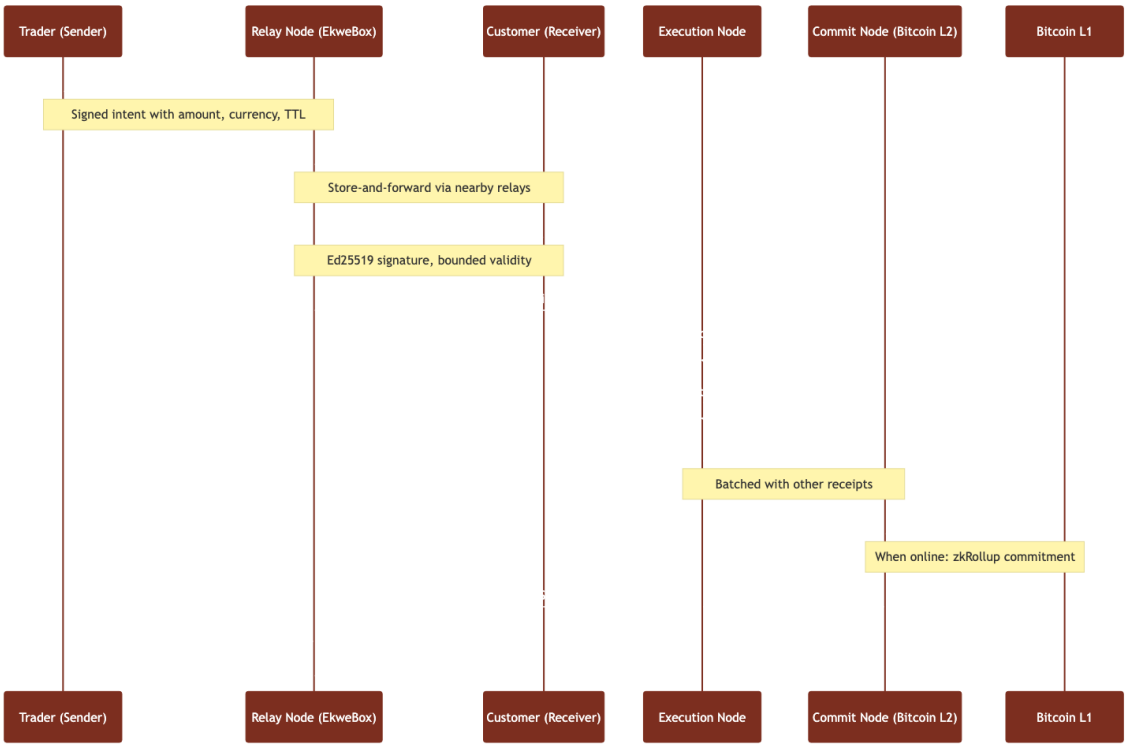


Figure 6: EkwePay offline transaction workflow.

Detailed steps:

1. **Initiation**: Trader creates a `PAYMENT.REQUEST` event on her phone (e.g., `{amount: 2500, currency: "NGN"}`).
2. **Relay**: Nearby EkweBox gossips the event over LoRa; other nodes forward via BLE/Wi-Fi.
3. **Delivery**: Customer's phone receives the request via BLE from a nearby relay.
4. **Confirmation**: Customer signs a `PAYMENT.CONFIRM` event, relayed back through the network.
5. **Execution**: An Execution Node validates the authorization, runs payment logic (WASM/zkVM), generates a proof, and queues a commitment.
6. **Settlement**: When any node regains Internet, proofs and receipts are batch-committed to the Bitcoin L2.

7. **Gas distribution**: Fees are split among the trader's integrator (originator), all relayers in the trace, the executor, and the commit node.

14.2 Offline Risk Controls

To control offline risk, EkwePay policies include:

- **Bounded transaction limits**: maximum offline transaction value per identity per time window.
- **Short validity windows**: signed intents expire, preventing indefinite deferral.
- **Delayed finality**: transactions are provisional until settlement confirmation.
- **Reputation staking**: nodes and users stake $EKWE against misbehavior.

14.2.1 Risk Bound Formalization

Let $L_{\max}$ be the maximum permitted offline exposure per identity. For a user u with outstanding unsettled transactions $\{t_1, \ldots, t_m\}$, the offline constraint is:

$$\sum_{j=1}^{m} v_j \leq L_{\max}(u) \quad \text{where } v_j = \text{value}(t_j) \text{ and } \text{age}(t_j) < \Delta_{\max} \tag{21}$$

The per-user limit $L_{\max}(u)$ is a function of stake and reputation:

$$L_{\max}(u) = L_0 + \kappa \cdot s_u \cdot \rho_u \tag{22}$$

where L_0 is a base limit, s_u is the user's staked $EKWE, ρ_u is reputation (Eq. 17), and κ is a governance-controlled leverage parameter. Transactions exceeding $L_{\max}$ or with $\text{age}(t_j) \geq \Delta_{\max}$ are rejected by honest relayers until settlement clears outstanding exposure.

The expected loss from double-spend attempts under this model is bounded by:

$$\mathbb{E}[\text{loss}] \leq L_{\max}(u) \cdot P_{\text{double-spend}} \cdot (1 - P_{\text{detect}}) \tag{23}$$

where P_{detect} increases with network density and decreases with partition duration, providing a tunable risk/availability tradeoff.

15. Hardware: EkweBox and Smartbox

15.1 EkweBox

The EkweBox is purpose-built hardware designed for permanent community relay infrastructure:

- **Radios**: LoRa transceiver + BLE 5.0 + Wi-Fi (AP + P2P mode).
- **Compute**: ARM-based SoC (e.g., Raspberry Pi CM4 or ESP32-S3 for lower cost).
- **Storage**: Local flash/SD for event cache and partial blockchain state.
- **Power**: Solar panel + battery + optional AC, designed for 24/7 operation in off-grid environments.
- **Interface**: USB/Serial for integration; optional display for diagnostics.

15.2 Smartbox

The Smartbox extends the EkweBox with mining capabilities:

- All EkweBox capabilities, plus:
- **Mining ASIC**: Interaction Combinator-based ASIC architecture for energy-efficient Bitcoin mining.
- **ZK co-processor**: hardware acceleration for proof generation.
- **Enhanced storage**: for archival node functionality and full blockchain state.

The Smartbox creates a compelling DePIN value proposition: a single device routes Ekwe events (earning $EKWE gas fees), mines Bitcoin (earning BTC), and provides community communication infrastructure, all powered by solar energy.

16. SDK and Developer Toolkit

Ekwe's developer ecosystem is built around a layered SDK architecture that enables third-party applications to leverage the protocol's offline-first dissemination, verifiable execution, and economic primitives without managing transport, routing, or settlement directly.

16.1 SDK Architecture

The Ekwe SDK consists of three tiers:

Table 3: Ekwe SDK tiers.

Tier	Language	Scope
Core SDK	Rust	Embeddable Edge Agent library: event creation, signing, subscription management, transport abstraction, local storage. Compiles to native, WASM, and C-FFI targets.
Mobile SDK	Kotlin / Swift	Native Android and iOS wrappers around Core SDK via FFI. Handles OS-level BLE/Wi-Fi permissions, background execution, and push notification bridging.
Web SDK	TypeScript	Browser and Node.js client for REST/WebSocket interaction with Edge Agent. Event builder, subscription API, and wallet integration.

16.2 Core API Surface

The Core SDK exposes a minimal, transport-agnostic API:

Listing 3: Core SDK interface (Rust).

```
pub trait EkweClient {
```

```
    // Event lifecycle
    fn create_event(payload: Payload, qos: QoSClass,
                    ttl: Duration) -> Result<EkweEvent>;
    fn sign_event(event: &mut EkweEvent,
                  key: &SigningKey) -> Result<()>;
    fn submit_event(event: EkweEvent) -> Result<EventId>;

    // Subscriptions
    fn subscribe(filter: SubscriptionFilter,
                 handler: EventHandler) -> Result<SubId>;
    fn unsubscribe(sub_id: SubId) -> Result<()>;

    // Payments (EkwePay)
    fn create_payment(amount: u64, currency: &str,
                      recipient: &PubKey) -> Result<EkweEvent>;
    fn confirm_payment(request: &EkweEvent,
                       key: &SigningKey) -> Result<EkweEvent>;

    // Node & wallet
    fn stake(amount: u64, role: NodeRole) -> Result<TxId>;
    fn balance() -> Result<Balance>;
    fn node_status() -> Result<NodeInfo>;
}
```

16.3 Transport Abstraction

Application developers interact exclusively with the event-level API. The SDK's transport layer automatically selects the optimal bearer based on availability and policy:

$$t^* = \arg\min_{t \in \mathcal{T}_{\text{avail}}} \left[c_{\text{energy}}(t) + c_{\text{latency}}(t) \cdot \mu(q) \right] \tag{24}$$

where $\mathcal{T}_{\text{avail}}$ is the set of currently available transports, $c_{\text{energy}}(t)$ and $c_{\text{latency}}(t)$ are normalized cost estimates, and $\mu(q)$ is the QoS multiplier (Eq. 8). For Class 2 events, the SDK may broadcast across multiple transports simultaneously.

16.4 Developer Workflow

A typical integration follows four steps:

1. **Initialize**: instantiate `EkweClient` with node configuration and key material.
2. **Subscribe**: register event handlers for relevant event types and topics.
3. **Emit**: create, sign, and submit events. The SDK handles routing, relay selection, and fee attachment.
4. **Settle**: for payment events, monitor settlement status via callback or polling.

16.5 Gas Sponsorship for dApps

Third-party applications (clinics, logistics platforms, alert systems) can pre-fund gas accounts to sponsor event delivery for their end users:

Listing 4: Gas sponsorship example.

```
// Integrator pre-funds a gas account
let gas_account = client.create_gas_account(
    initial_balance: 10_000,  // $EKWE
    authorized_topics: vec!["clinic.alerts", "clinic.logistics"],
    rate_limit: RateLimit::PerMinute(100),
)?;

// End-user events are gas-free at the application layer
let alert = client.create_event(
    payload: Payload::Data(patient_alert),
    qos: QoSClass::Critical,
    sponsor: gas_account.id,
)?;
```

This mirrors the integrator-sponsored gas model (Section 10.2), enabling zero-cost end-user experiences while maintaining protocol-level economic accountability.

17. Security and Privacy

17.1 Identity and Authentication

Nodes are bound to Ed25519 cryptographic identities. All events are signed at origin and verified at each relay and execution hop. Local user authentication employs JWT or passcode-based mechanisms.

17.2 Replay, Spam, and Flooding Resistance

- **Replay resistance**: event identifiers, monotonic nonces, and bounded TTL.
- **Spam resistance**: minimum fee policies and rate limits at relays; higher QoS classes require stronger fee commitments.
- **Anti-flooding**: TTL bounds prevent indefinite message propagation.

17.3 Sybil Resistance

Fee-only resistance is insufficient in adversarial environments. Ekwe employs a layered approach:

- **Stake requirements**: nodes must stake $EKWE proportional to their role privileges.
- **Proof-of-availability**: tracked uptime and relay metrics.
- **Reputation signals**: historical fulfillment accuracy.
- **Governance thresholds**: dynamically adjusted based on network conditions.

17.4 Privacy and Metadata Minimization

Relay traces support reward attribution but introduce metadata leakage risks. Ekwe treats privacy-preserving accounting as a first-class design concern:

- **ZK-based reward claims**: proving relay participation without revealing the full trace.
- **Blinded relay sets**: aggregated receipts that obscure individual node contributions.
- **End-to-end encryption**: event payloads are encrypted; only addressing metadata is visible to relays.
- **Gas blindness**: fees are computed on resource usage, not message content, due to ZK properties.

18. Implementation Blueprint and Evaluation Methodology

18.1 Reference Implementation

Table 4: Implementation components.

Component	Technology	Description
Edge Agent	Rust/Go	Cross-platform, embeddable runtime with event store, verifier, forwarder, and subscription manager
Mobile App	Flutter + Native SDK	Runs Edge Agent; provides UX for EkwePay, contact book, QR sync
Radio Stack	Hardware APIs	LoRaWAN (Semtech), BLE (ESP32/phone), Wi-Fi Direct
Execution VM	WASM / zkVM	Deterministic execution with receipt generation [14]
Mini-Chain	Embedded DB	SQLite-based local chain or zk-rollup buffer
Rules Engine	Embedded	Rule-based event router for cache-first logic

18.2 Evaluation Metrics

The following metrics are formally defined for benchmarking:

1. **Delivery ratio** $\mathcal{D}(T)$: fraction of targeted nodes receiving an event within time T.

$$\mathcal{D}(T) = \frac{|\{n \in \mathcal{S} : t_{\text{recv},n} \leq t_{\text{emit}} + T\}|}{|\mathcal{S}|} \tag{25}$$

   where $\mathcal{S}$ is the set of subscribed (targeted) nodes.

2. **End-to-end latency** ℓ_n: for each receiving node n, $\ell_n = t_{\text{recv},n} - t_{\text{emit}}$.

3. **Overhead ratio** Ω: total transmissions per delivered event.

$$\Omega = \frac{\sum_{\text{all nodes}} \text{tx}_i}{\sum_{\text{events}} \mathcal{D}(T) \cdot |\mathcal{S}|} \tag{26}$$

4. **Energy cost** E_{event}: radio-on time per delivered event, normalized by transport.

$$E_{\text{event}} = \sum_{t \in \mathcal{T}} P_t \cdot \tau_{t,\text{active}} \tag{27}$$

   where P_t is the power draw of transport t and $\tau_{t,\text{active}}$ is the active radio time.

5. **Economic efficiency** η_{econ}: reward per delivered byte versus spam cost.

$$\eta_{\text{econ}} = \frac{R_{\text{relayer, total}}}{\text{bytes}_{\text{delivered}}} \quad \text{vs.} \quad C_{\text{spam}} = F(e_{\min}) \cdot \frac{1}{\mathcal{D}(T)} \tag{28}$$

18.3 Evaluation Scenarios

1. **Rural partition**: simulated villages with sparse nodes and intermittent backhaul; compare QoS classes and replication factors.
2. **Urban congestion**: high node density with adversarial spam; evaluate fee-based throttling.
3. **Mixed-transport**: BLE/Wi-Fi clusters bridged by LoRaWAN relays; quantify cross-transport propagation.

18.4 Baselines

1. **Centralized push**: APNs/FCM delivery degrades to zero without Internet [2].
2. **DTN baseline**: store-and-forward overlay without incentives [1].
3. **DePIN coverage**: incentive-driven coverage (Helium-style) without general event execution [15].

19. Roadmap

Table 5: Ekwe Protocol development roadmap.

Phase	Version	Milestones
Foundation	v0.1	System design document; core protocol specification; Edge Agent prototype (Rust)
Tokenomics	v0.2	$EKWE token design; staking model; gas parameter calibration
Hardware	v0.3	EkweBox hardware specification; Smartbox design with Interaction Combinator ASIC integration
Developer	v0.4	App SDKs (Android/iOS); developer portal; third-party dApp integration APIs
Testnet	v0.5	Public testnet; controlled rural and urban pilot deployments
Mainnet	v1.0	Production-ready zk-Execution engine; Bitcoin L2 rollup chain; EkwePay launch
Scale	v2.0	Multi-region sync bridges; priority emergency lanes; cross-chain interoperability

20. Limitations and Open Problems

1. **Offline double-spend**: purely offline payments require careful risk bounding (limits, time windows, and eventual settlement). Fully offline double-spend prevention for high-value payments remains outside scope without additional synchronization or trusted hardware assumptions.
2. **Sybil resistance**: permissionless relaying needs robust anti-Sybil mechanisms beyond fees alone (stake, proof-of-coverage, reputation). The optimal combination remains an empirical question.
3. **Privacy vs. accountability**: relay traces aid reward splitting but can leak metadata; privacy-preserving accounting (ZK-based reward claims, blinded relay sets) remains an active research direction.
4. **Mobile OS constraints**: background relaying is limited by platform policies (iOS background execution, Android Doze mode); dedicated Smartboxes are required for high-availability deployments.
5. **Governance calibration**: fee parameters $(\alpha, \beta, \gamma, \delta)$ and reward splits require empirical tuning under real deployment conditions.
6. **Consensus finalization**: the optimal consensus mechanism for challenged-network environments with intermittent global synchronization remains an open design question.

21. Conclusion

Ekwe Protocol is specified as a decentralized, event-driven edge substrate that generalizes "push notifications" into a resilient, offline-first dissemination and execution layer. Drawing inspiration from the Igbo Ekwe drum, a community-operated, decentralized communication instrument, the protocol reimagines critical infrastructure for environments where connectivity cannot be assumed.

The protocol differs from centralized push systems by removing dependence on always-on Internet brokers, and it differs from USSD-based financial access by enabling operation beyond the operator signaling plane, including scenarios where cellular coverage is absent or unreliable [2, 4]. Ekwe integrates DTN-style store-and-forward routing [1], multi-radio relaying across BLE, Wi-Fi Direct, and LoRaWAN [8, 9, 7], TON-inspired gas accounting with reward splitting [10, 11], verifiable execution via zkVM [14], and Bitcoin L2 anchoring for settlement [12, 13].

The dual-token model ($EKWE for protocol utility and BTC from Smartbox mining) creates aligned incentives for deploying and maintaining physical infrastructure. The flagship application, EkwePay, demonstrates how offline-first financial transactions can operate through community-provisioned relay networks with eventual settlement on Bitcoin.

The next research and engineering steps include: a reference implementation of the Edge Agent; controlled evaluation across rural and urban deployment scenarios; formalization of privacy-preserving incentive accounting; and production of the EkweBox and Smartbox hardware platforms.

The drum speaks. The network listens. The people are sovereign.

References

[1] V. Cerf *et al.*, "Delay-Tolerant Networking Architecture," RFC 4838, IETF, 2007. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc4838.html`

[2] Apple Developer Documentation, "Establishing a Token-Based Connection to APNs," Apple Push Notification service. [Online]. Available: `https://developer.apple.com/documentation/usernotifications/establishing-a-token-based-connection-to-apns`

[3] Google Firebase Documentation, "Send a Message Using the FCM HTTP v1 API," Firebase Cloud Messaging. [Online]. Available: `https://firebase.google.com/docs/cloud-messaging/send/v1-api`

[4] ETSI (3GPP), "TS 122 090: Unstructured Supplementary Service Data (USSD); Stage 1," v18.0.1. [Online]. Available: `https://www.etsi.org/deliver/etsi_ts/122000_122099/122090/18.00.01_60/ts_122090v180001p.pdf`

[5] ETSI (3GPP), "TS 123 090: Unstructured Supplementary Service Data (USSD); Stage 2," v7.0.0. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/123000_123099/123090/07.00.00_60/ts_123090v070000p.pdf

[6] GSMA, "Beyond USSD: How Offline NFC Transactions Can Drive Mobile Money Use," 2025. [Online]. Available: https://www.gsma.com/solutions-and-impact/connectivity-for-good/mobile-for-development/blog/beyond-ussd-how-offline-nfc-transactions-can-drive-mobile-money-use/

[7] LoRa Alliance, "TS001-1.0.4 LoRaWAN® L2 1.0.4 Specification," 2023. [Online]. Available: https://resources.lora-alliance.org/technical-specifications/ts001-1-0-4-lorawan-l2-1-0-4-specification

[8] Bluetooth SIG, "Bluetooth Mesh Profile 1.0.1," Adopted Specification. [Online]. Available: https://www.bluetooth.com/specifications/specs/mesh-profile-1-0-1/

[9] Android Developers, "Wi-Fi Direct (Wi-Fi P2P) Overview." [Online]. Available: https://developer.android.com/develop/connectivity/wifi/wifip2p

[10] The Open Network (TON), "TON Whitepaper," 2021. [Online]. Available: https://ton.org/whitepaper.pdf

[11] TON Documentation, "Transaction Fees." [Online]. Available: https://docs.ton.org/foundations/fees

[12] M. Qi *et al.*, "SoK: Bitcoin Layer Two (L2)," *ACM Computing Surveys*, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3763232

[13] R. Linus, "BitVM2: Bridging Bitcoin to Second Layers," 2024. [Online]. Available: https://bitvm.org/bitvm_bridge.pdf

[14] RISC Zero, "Proof System Overview," Developer Documentation. [Online]. Available: https://dev.risczero.com/proof-system/

[15] Helium, "Helium Whitepaper." [Online]. Available: https://whitepaper.helium.com/

[16] Messari, "Understanding DePIN," 2023–2024. [Online]. Available: https://messari.io/copilot/share/understanding-depin-26867bfa-d608-49bf-a2c3-0f14d76f2d6a